TUESDAY(17/FEB/2026) - INTRODUCTION TO C-SHARP **[WEEK3]**

Introduction To C#

***Visual Studio**

-**C#** is **case sensitive**, variables should all be in lower case.

Here is the introductory code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
//directive that allows system console object class to be written without prefix

namespace FirstProj {
   class Program { //creating our own program properties and method
      static void Main(string[] args) //main method where the code is written
      {
         Console.Title = "Vaiables";

         char letter; letter = 'A';
         int number; number = 100;
         float body = 98.6f;
         double pi = 3.14;
         decimal sum = 1000.00m;
         bool flag = false;
         string text = "Mbulelo is amazing";
```

```
        //VARIABLE (Placeholder): data-type then variable name

        Console.WriteLine("Hello, Mbulelo!");//statement that calls the method of class to output the text in parenthesis

        Console.WriteLine("char letter:\t" + letter);

        Console.WriteLine("int number:\t" + number);

        Console.WriteLine("float body:\t" + body);

        Console.WriteLine("double pi:\t" + pi);

        Console.WriteLine("decimal sum:\t" + sum);

        Console.WriteLine("bool flag:\t" + flag);

        Console.WriteLine("string text:\t" + text);

        Console.ReadKey();

        //Console.ReadKey(); //press any key to end program



    }

}


}
```

RESULT OF THE ABOVE CODE:
Hello, Mbulelo! char letter: A int number: 100 float body: 98,6 double pi: 3,14 decimal sum: 1000,00 bool flag: False string text: Mbulelo is amazing

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
//directive that allows system console object class to be written without prefix

namespace FirstProj {
    0 references
    class Program { //creating our own program properties and method
        0 references
        static void Main(string[] args) //main method where the code is written
        {
            Console.Title = "Vaiables";

            char letter; letter = 'A';
            int number; number = 100;
            float body = 98.6f;
            double pi = 3.14;
            decimal sum = 1000.00m;
            bool flag = false;
            string text = "Mbulelo is amazing";

            //VARIABLE (Placeholder): data-type then variable name
            Console.WriteLine("Hello, Mbulelo!");//statement that calls the method of class to output the text in parenthesis
            Console.WriteLine("char letter:\t" + letter);
```

How to accept value from user:

```
class Program// creating our own program properties and method
{
    static void Main(string[] args)// main method, were we are writing
    {
        Console.Title = "Input";


        Console.WriteLine(" Please Enter Your Name:");
        string name = Console.ReadLine();
        Console.WriteLine(" Welcome" + name + "!");

        Console.ReadKey();

    }
```

Adding numbers:

```
Console.WriteLine(" Please Enter A Number: ");

double num = Convert.ToDouble(Console.ReadLine());

Console.WriteLine(" Now Enter Another Number: ");

double sum = num + Convert.ToDouble(Console.ReadLine());

Console.WriteLine(" Total = " + sum);
```

ARRAYS

*First element starts at 0.

Datatype [] arrayname = new datatype[size];

e.g. **Int[] nums = new int[5]**

Array-name [element number] = value;

A = [1,2,3,4,5,6,7,8,9,10] {} or A could be =[A,B,C,D,E,F,G,H,I,J] (These are the elements of an array).

*The **index** for this array would be starting from 0

i.e. [0,1,2,3,4,5,6,7,8,9]

For example if we want to find the 4th element of Array A using the index that would be index 3 for the numbers, and C for the alphabet.

```
using System;

using System.Collections.Generic;

using System.Linq;

using System.Text;

using System.Threading.Tasks;

//directive that allows system console object class to be written without prefix


namespace FirstProj {

   class Program { //creating our own program properties and method

      static void Main(string[] args) //main method where the code is written

      {

         Console.Title = "Arrays";


         //CREATED A STRING ARRAY

         string[] cars = new string[3] { "BMW", "FORD", "'VW" };
```

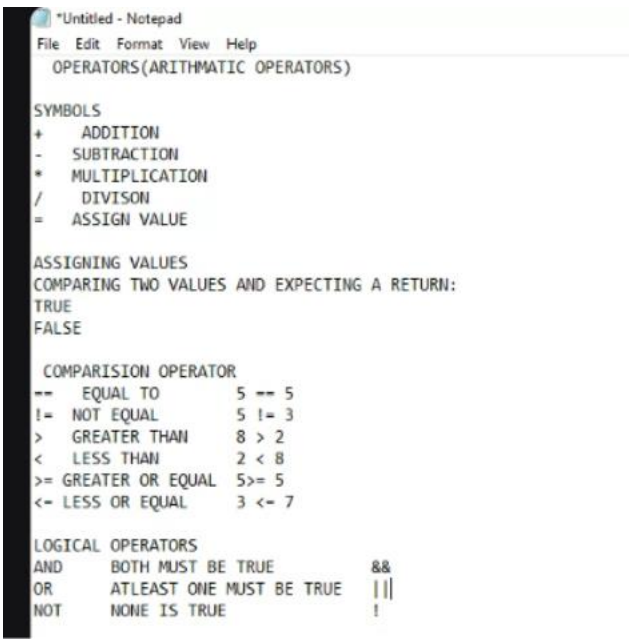
```
        //CREATE A 2 DIMENSIONAL ARRAY
        int[,] coords = new int[2, 3] { { 1, 2, 3 }, { 4, 5, 6 } };

        //DISPLAY OUTPUT WITH STORED ARRAY ELEMENTS VALUES

        Console.WriteLine(" Second Car: " + cars[1]);
        Console.WriteLine(" X1,Y1 = " + coords[0, 0]);//this should be 1
        Console.WriteLine(" X2,Y3 = " + coords[1, 2]);//this should be 6

    }
}

}
```

* C-Sharp Activity (15:15)

WEDNESDAY(18/FEB/2026) - **[WEEK3]**

Exercise in the C-Sharp Activities

THURSDAY (19/FEB/2026) - **[WEEK3]**



```
*Untitled - Notepad
File  Edit  Format  View  Help
  OPERATORS(ARITHMATIC OPERATORS)

SYMBOLS
+    ADDITION
-   SUBTRACTION
*   MULTIPLICATION
/    DIVISON
=   ASSIGN VALUE

ASSIGNING VALUES
COMPARING TWO VALUES AND EXPECTING A RETURN:
TRUE
FALSE

 COMPARISION OPERATOR
==   EQUAL TO        5 == 5
!=  NOT EQUAL        5 != 3
>   GREATER THAN     8 > 2
<   LESS THAN        2 < 8
>= GREATER OR EQUAL  5>= 5
<= LESS OR EQUAL     3 <= 7

LOGICAL OPERATORS
AND     BOTH MUST BE TRUE          &&
OR      ATLEAST ONE MUST BE TRUE   ||
NOT     NONE IS TRUE               !
```

OPERATORS(ARITHMATIC OPERATORS)

SYMBOLS

+ ADDITION

- SUBTRACTION

* MULTIPLICATION

/ DIVISON

= ASSIGN VALUE

ASSIGNING VALUES

COMPARING TWO VALUES AND EXPECTING A RETURN:

TRUE

FALSE

COMPARISION OPERATOR

== EQUAL TO 5 == 5

!= NOT EQUAL 5 != 3

> GREATER THAN 8 > 2

< LESS THAN 2 < 8

>= GREATER OR EQUAL 5>= 5

<= LESS OR EQUAL 3 <= 7

LOGICAL OPERATORS

AND BOTH MUST BE TRUE &&

OR ATLEAST ONE MUST BE TRUE ||

NOT NONE IS TRUE !

MONDAY (23/FEB/2026) - **[WEEK4]**

-Variables

-Constants:

Constants

If you don't want others (or yourself) to overwrite existing values, you can add the `const` keyword in front of the variable type.

This will declare the variable as "constant", which means unchangeable and read-only:

Example

```
const int myNum = 15;
myNum = 20; // error
```

C# Type Casting

Type casting is when you assign a value of one data type to another type.

In C#, there are two types of casting:

- **Implicit Casting** (automatically) - converting a smaller type to a larger type size
  `char` -> `int` -> `long` -> `float` -> `double`

- **Explicit Casting** (manually) - converting a larger type to a smaller size type
  `double` -> `float` -> `long` -> `int` -> `char`

The datatype you want to convert to:

```
int x = 12;

double y = 45.67d;

int a = (int) y;
```

We say default = implicit, that's what it means:
[Implicit Casting/Conversion]

Implicit Casting

default

Implicit casting is done automatically when passing a smaller size type to a larger size type:

Example

```
int myInt = 9;
double myDouble = myInt;      // Automatic casting: int to double

Console.WriteLine(myInt);      // Outputs 9
Console.WriteLine(myDouble);   // Outputs 9
```

WHAT WE GOING TO DO (2 WEEKS)

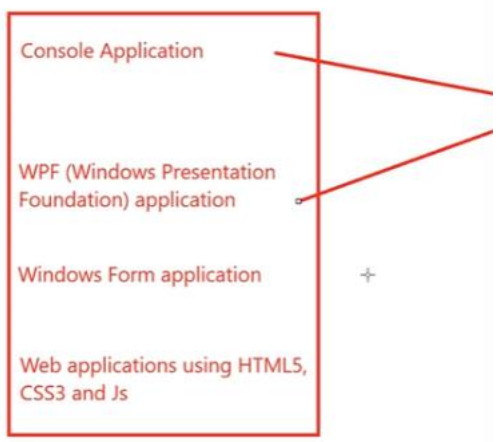
-Monday to Wednesday (Console Application)

-Thursday we doing the U.I.

-WPF (IS the new thing)

-Next Week (Windows Form Application) - is the old thing

-Building Web Applications (HTML5, CSS3 & Javascript)



*Class Activity In Emails (Quiz and Tasks From Francis)

LOOPS

While Loop

C# While Loop

The `while` loop loops through a block of code as long as a specified condition is `True`:

Syntax

```
while (condition)
{
  // code block to be executed
}
```

In the example below, the code in the loop will run, over and over again, as long as a variable (i) is less than 5:

Example

```
int i = 0;
while (i < 5)
{
  Console.WriteLine(i);
  i++;
}
```

```
int a = 0;

while (a < 5)
{

    Console.WriteLine(a);
    a++;
}
```

<u>Do-while</u>

(Will get executed once before checking if the condition is true, then continue execution as long as the condition is true).

The Do/While Loop

The `do/while` loop is a variant of the `while` loop. This loop will execute the code block once, before checking if the condition is true, then it will repeat the loop as long as the condition is true.

Syntax

```
do
{
  // code block to be executed
}
while (condition);
```

The example below uses a `do/while` loop. The loop will always be executed at least once, even if the condition is false, because the code block is executed before the condition is tested:

Example

```
int i = 0;
do
{
  Console.WriteLine(i);
  i++;
}
while (i < 5);
```

<u>*For Loop</u>

C# For Loop

When you know exactly how many times you want to loop through a block of code, use the `for` loop instead of a `while` loop:

Syntax

```
for (statement 1; statement 2; statement 3)
{
  // code block to be executed
}
```

Statement 1 is executed (one time) before the execution of the code block.

Statement 2 defines the condition for executing the code block.

Statement 3 is executed (every time) after the code block has been executed.

The example below will print the numbers 0 to 4:

Controlling a Loop with a break statement:

```
for (int a = 0; a <= 5; a++)
{
    if (a==3)
    {
        break;
    }

    Console.WriteLine(a);
}
```

\

When it gets to 3, it skips because of continue

```
for (int a = 0; a <= 5; a++)
{
    if (a==3)
    {
        continue; //skip
    }

    Console.WriteLine(a);
}
```

<u>*FOR-EACH LOOP</u>

On Arrays we use it most (For each item in a collection)

The foreach Loop

There is also a `foreach` loop, which is used exclusively to loop through elements in an **array** (or other data sets):

Syntax

```
foreach (type variableName in arrayName)
{
  // code block to be executed
}
```

The following example outputs all elements in the **cars** array, using a `foreach` loop:

Example

```
string[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
foreach (string i in cars)
{
  Console.WriteLine(i);
}
```

Volvo
BMW

Just A Reminder:

Create an Array

Arrays are used to store multiple values in a single variable, instead of declaring separate variables for each value.

To declare an array, define the variable type with **square brackets**:

```
string[] cars;
```

We have now declared a variable that holds an array of strings.

To insert values to it, we can use an array literal - place the values in a comma-separated list, inside curly braces:

```
string[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
```

To create an array of integers, you could write:

```
int[] myNum = {10, 20, 30, 40};
```

-Arrays are fixed in size

SO, here's how to use Arrays sorting by using the ForEach Loop:

Sort an Array

There are many array methods available, for example `Sort()`, which sorts an array alphabetically or in an ascending order:

Example

Get you

```
// Sort a string
string[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
Array.Sort(cars);
foreach (string i in cars)
{
  Console.WriteLine(i);
}
```

asc
Desc

Error Handling:

C# Exceptions

When executing C# code, different errors can occur: coding errors made by the programmer, errors due to wrong input, or other unforeseeable things.

When an error occurs, C# will normally stop and generate an error message. The technical term for this is: C# will throw an **exception** (throw an error).

C# try and catch

The `try` statement allows you to define a block of code to be tested for errors while it is being executed.

The `catch` statement allows you to define a block of code to be executed, if an error occurs in the try block.

The `try` and `catch` keywords come in pairs:

Using the Try – Catch Block:

```
int a = 10;
int b = 0;

try
{
    int Result = a / b;


}
catch (Exception ex)
{
    Console.WriteLine(ex.ToString());
}
```

```
int a = 10;

int b = 0;


try

{

   int results = a / b;

}

catch (Exception ex) {

Console.WriteLine(ex.ToString());

}
```

Else

```
int a = 10;

int b = 10;


try

{

   int results = a / b;

   Console.WriteLine(results);

}

catch (Exception ex) {
```

Console.WriteLine(ex.ToString());

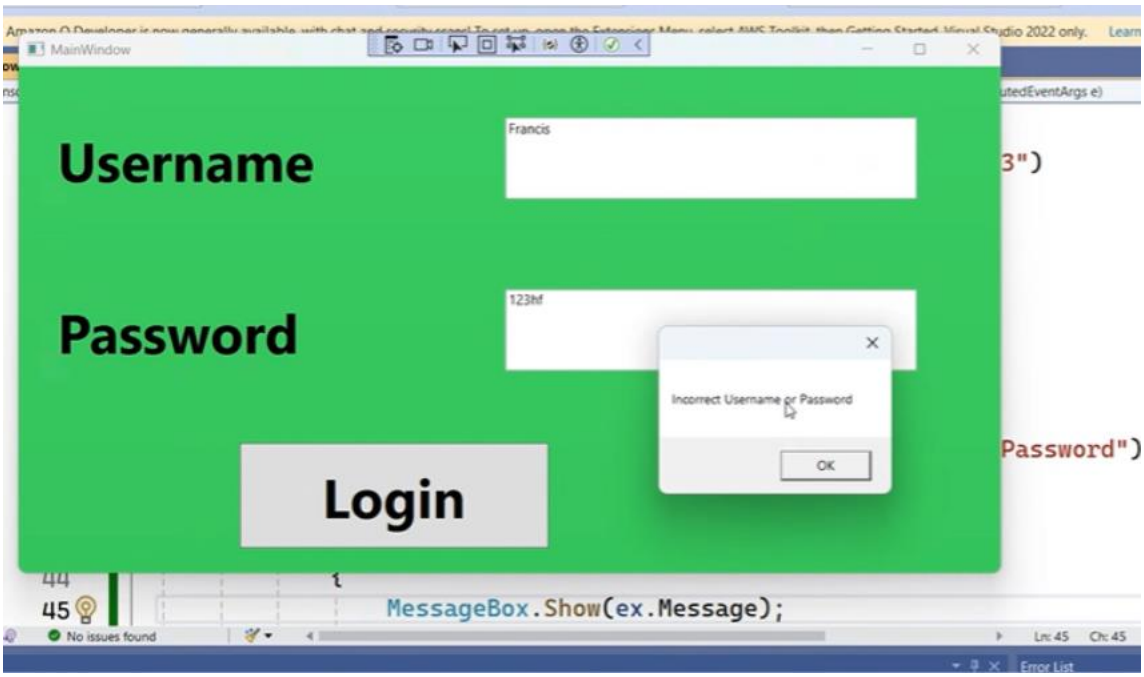
}

Here's A Full example:

```
string username = "Francis";
string password = "123";

try
{
    if (username == "Francis" && password == "123")
    {
        Console.WriteLine("Login Successful");
    }
    else
    {
        Console.WriteLine("Incorrect Username or Password");
    }

}
catch (Exception ex)
{
    Console.WriteLine(ex.ToString());
}
```

class System.String
Represents text as a sequence of UTF-16 code units.

<u>BUILDING A WINDOWS FORM WITH THE TRY-CATCH</u>




```
string Password = txtpassword.Text;

try
{
    if (Username == "Francis" & Password == "123")
    {
        MessageBox.Show("Login Successful");

        WelcomeAdmin ad = new WelcomeAdmin();
        ad.Show();
    }
    else
    {
        MessageBox.Show("Incorrect Username or Password");
    }
}
catch (Exception ex)
{
    MessageBox.Show(ex.Message);
}
```

FINALLY BLOCK

-This is the block that will always run

-Now in the finally block, there is a Close () function closes the form.

```
        else
        {
            MessageBox.Show("Incorrect Username or Password");
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message);
    }
    finally
    {
        Close();
    }
}
```

Here's a type of exception (Error)- Array out of bounds:

Consider the following example, where we create an array of three integers:

This will generate an error, because **myNumbers[10]** does not exist.

```
int[] myNumbers = {1, 2, 3};
Console.WriteLine(myNumbers[10]); // error!
```

The error message will be something like this:

```
System.IndexOutOfRangeException: 'Index was outside the bounds of the array.'
```

<u>THE THROW BLOCK:</u>

The throw keyword

The `throw` statement allows you to create a custom error.

The `throw` statement is used together with an **exception class**. There are many exception classes available in C#: `ArithmeticException`, `FileNotFoundException`, `IndexOutOfRangeException`, `TimeOutException`, etc:

Example

```
static void checkAge(int age)
{
  if (age < 18)
  {
    throw new ArithmeticException("Access denied - You must be at least 18 years old.");
  }
  else
  {
    Console.WriteLine("Access granted - You are old enough!");
  }
}

static void Main(string[] args)
{
  checkAge(15);
}
```

```
string username = "Francis";

string password = "123";


try

{

    if (username == "Francis" && password == "123")

    {

        Console.WriteLine("Login Successful");

    }

    else

    {

        Console.WriteLine("Incorrect Username or Password");

    }


}

catch (Exception ex)

{
```

```
    Console.WriteLine(ex.ToString());
}
```

TUESDAY (24/FEB/2026) - **[WEEK4]**

Namespace works with System.IO (Files), we call them packages in Java

Working With Files

built-in Functions -

The `File` class from the `System.IO` namespace, allows us to work with files:

Example

```
using System.IO;  // include the System.IO namespace

File.SomeFileMethod();  // use the file class with methods
```

Different File Functions:

(They exist in the Namespace [Packages])

The `File` class has many useful methods for creating and getting information about files. For example:

Method	Description
`AppendText()`	Appends text at the end of an existing file
`Copy()`	Copies a file
`Create()`	Creates or overwrites a file
`Delete()`	Deletes a file
`Exists()`	Tests whether the file exists
`ReadAllText()`	Reads the contents of a file
`Replace()`	Replaces the contents of a file with the contents of another file
`WriteAllText()`	Creates a new file and writes the contents to it. If the file already exists, it will be overwritten.

*Using System.IO

(Writing Contents to a File)

```
using System.IO;

string writeText = "Welcome to todays class";

File.WriteAllText("francis.txt", writeText);
```

(Function to Read A File):

```
string readText = File.ReadAllText("francis.txt");
Console.WriteLine(readText);
```

Real-world Example:

```
bool fileExist = false;

string thePath = Environment.GetFolderPath(Environment.SpecialFolder.MyPictures);

string theFile = thePath + @"\Lizaam.txt";

fileExist = File.Exists(theFile);
```

```
try
{


    if (fileExist)
    {
        Console.WriteLine("Tell us The file Exists");

    }
    else
    {
        Console.WriteLine("The file does not exist, let me create it");
        File.Create(theFile);
    }
    if (fileExist)
    {
        Console.WriteLine("The file was created on {0}", File.GetCreationTime(theFil
        Console.WriteLine("It was last accessed on {0}", File.GetLastAccessTime(theF
```

Here is the full code:

using System;

using System.Collections.Generic;

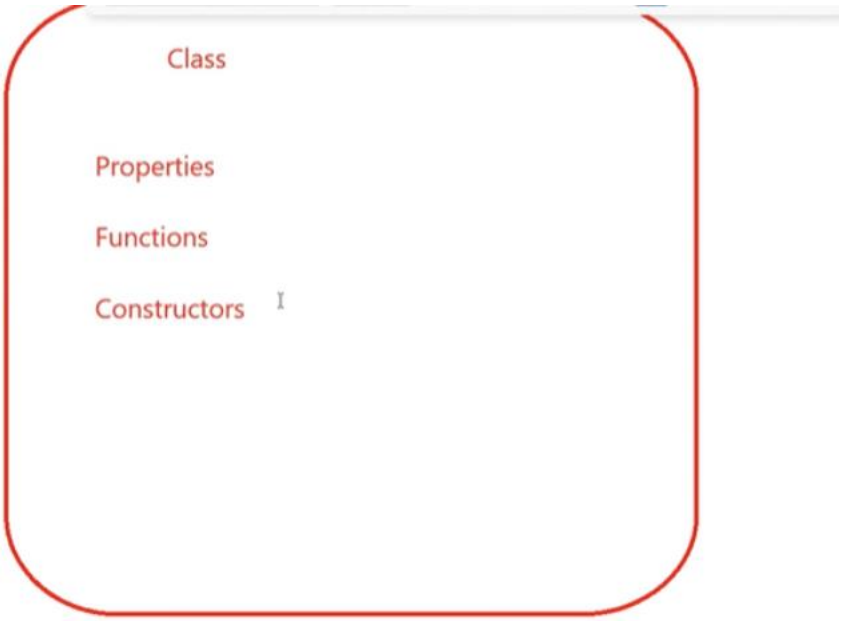
```
using System.Linq;
using System.Text;
using System.Threading.Tasks;



using System.IO;  //you need these namesapce to work with files//


namespace Serialization
{
   class Program
   {
      static void Main(string[] args)
      {



         bool fileExist = false;


         string thePath = Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments);


         string theFile = thePath + @"\Jennifer.txt";


         fileExist = File.Exists(theFile);


         try
         {
```

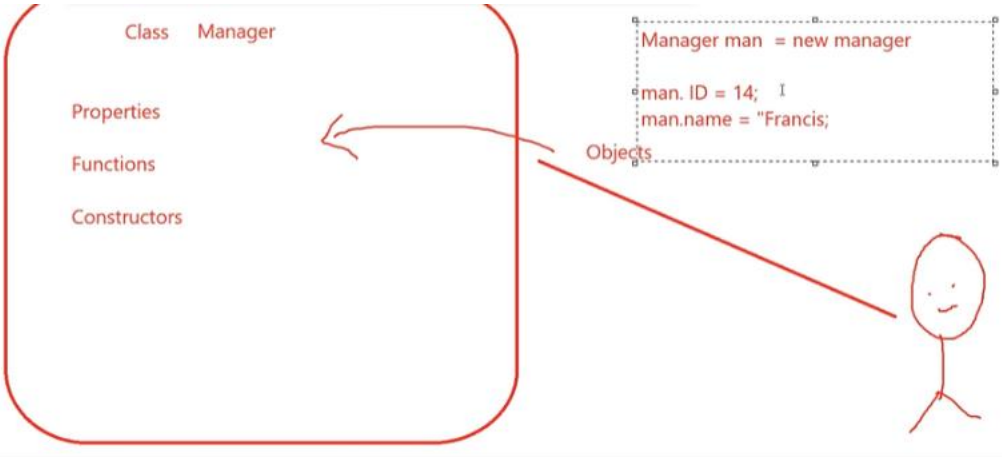
```
            if (fileExist)
            {
                Console.WriteLine("Tell us The file Exists");


            }
            else
            {
                Console.WriteLine("The file does not exist, let me create it");
                File.Create(theFile);
            }
            if (fileExist)
            {
                Console.WriteLine("The file was created on {0}", File.GetCreationTime(theFile));
                Console.WriteLine("It was last accessed on {0}", File.GetLastAccessTime(theFile));



                // Console.WriteLine("Moving the file....");
                //File.Move(theFile, thePath + @"\newfile.txt");  //this moves the file from its existing path
and create a new file and change the old file name//
            }


        }
        catch (Exception ex)


        {
            Console.WriteLine(ex.Message);
        }
        Console.ReadLine();
```

```
    }
  }
}
```

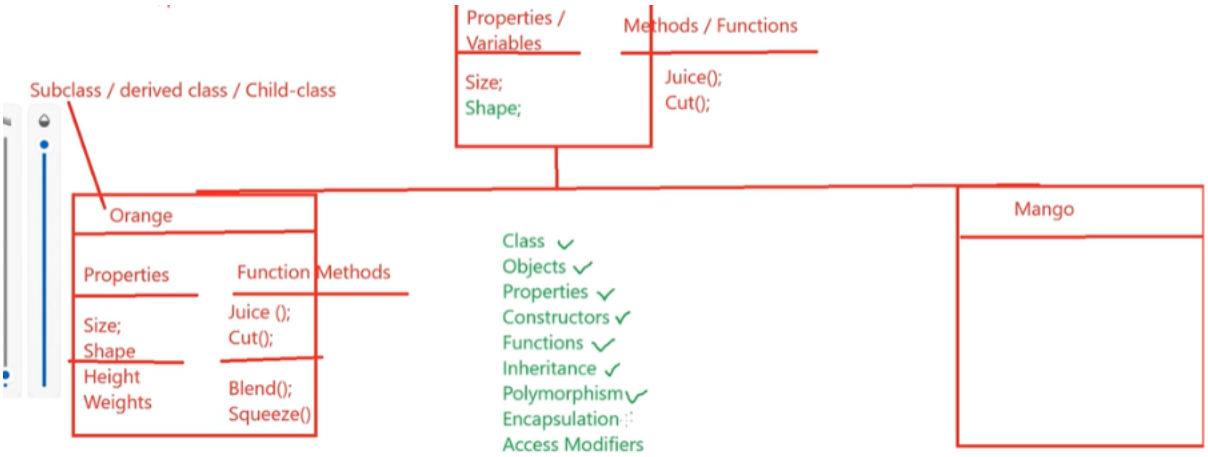
*Email Class Exercise With Francis.

<u>OBJECT ORIENTED PROGRAMMING</u>





- Class is a blueprint/template that defines an object
- Object (self-contained data structure) or is a container, Instance of A class.



This is the Class “Fruits”:

```
namespace Prj_DemoAfrovats
{
    2 references
    public class Fruits
    {

    }

}
```

“francis” is the object of the Class “Fruits”.

```
using Prj_DemoAfrovats;

Fruits francis = new Fruits();

francis.ID = 12;
```

Properties of a class: they're the variables of the class

```
public class Manager
{
    0 references
    int ID { get; set; }
    0 references
    string Name { get; set; }
    0 references
    string Surname { get; set; }
    0 references
    string Address { get; set; }


    decimal Salary {  get; set; }
}
```

<u>CONSTRUCTORS</u>

- Initialize values to your class properties
- Constructors have the same name with the class:

```
public Manager(int ID, string Name, string Surname, str
{
    ID = 123;
    Name = "Francis";
    Surname = "Anosike";
    Address = "Midrand";
    Salary = 25000.00m;
}
```

(This is a constructor of the Manager Class).

- Not a must to use these, because the compiler will create one for you

use to create an object /instance of a class

use to initialize values to class properties

it always have the same name as a class.

*__Function__ represents the behaviours of a class

```
0 references
public string getUserdetails(string Name, string Usernam
{
    return Name + Surname;
}
```

```
0 references
public string getUserdetails(string Name, string Usernam
{
    return Name + Surname;
}
0 references
public int getUserID(int ID)
{
    return ID;
}
0 references
```

(These are functions)

```
public string getMangerdetails(string Name, string Username)
{
    return Name + Surname;
}
0 references
public int getManagerID(int ID)
{
    return ID;
}
0 references
public decimal getManagerSalary(decimal Salary)
{
    return Salary;
}
```

You can add parameters in a function:

```
using Prj_DemoAfrovats;

Manager man = new Manager();

man.getMangerdetails();
```

```
using Prj_DemoAfrovats;

Manager man = new Manager();

int Result = man.getManagerID(123);

Console.WriteLine(Result);
```

This is inside the Class:

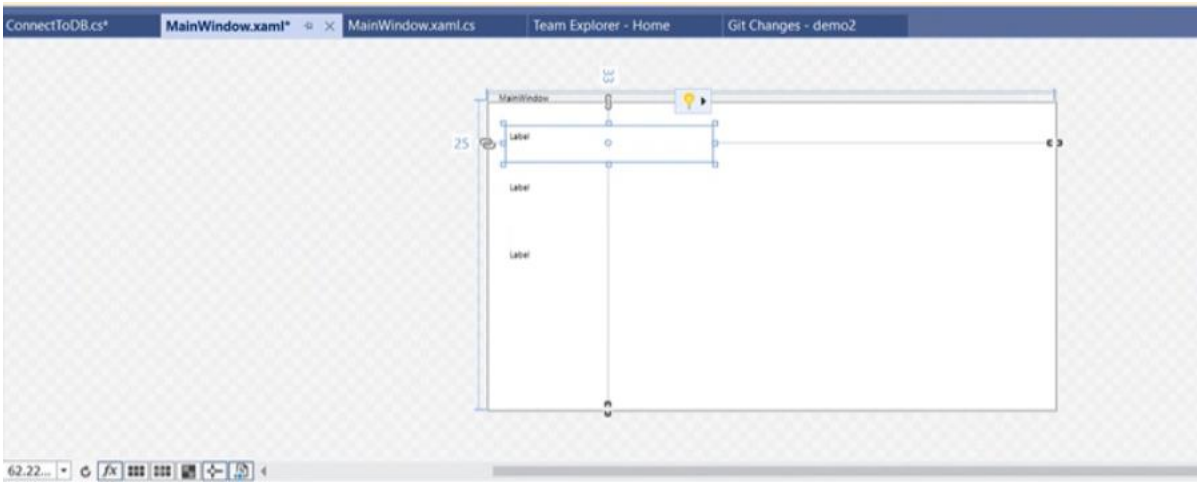
```
public string getMangerdetails(string Name, string Username)
{
    return Name + Surname;
}
1 reference
public int getManagerID(int ID)
{
  return ID;
}
0 references
public decimal getManagerSalary(decimal Salary)
{
    return Salary;
}
```

Outside the class we can use the object man to retrieve data from inside the class...

```
using Prj_DemoAfrovats;

Manager man = new Manager();

int Result = man.getManagerID(123);

Console.WriteLine(Result);
```

(local variable) int Result

Practical Example:



```
namespace WpfApp2
{
    0 references
    internal class ConnectToDB
    {


        0 references
        public void RegisterStudent(int ID, string Name,)
        {

        }
    }
}
```

C# Constructors

Constructors

A constructor is a **special method** that is used to initialize objects. The advantage of a constructor, is that it is called when an object of a class is created. It can be used to set initial values for fields:

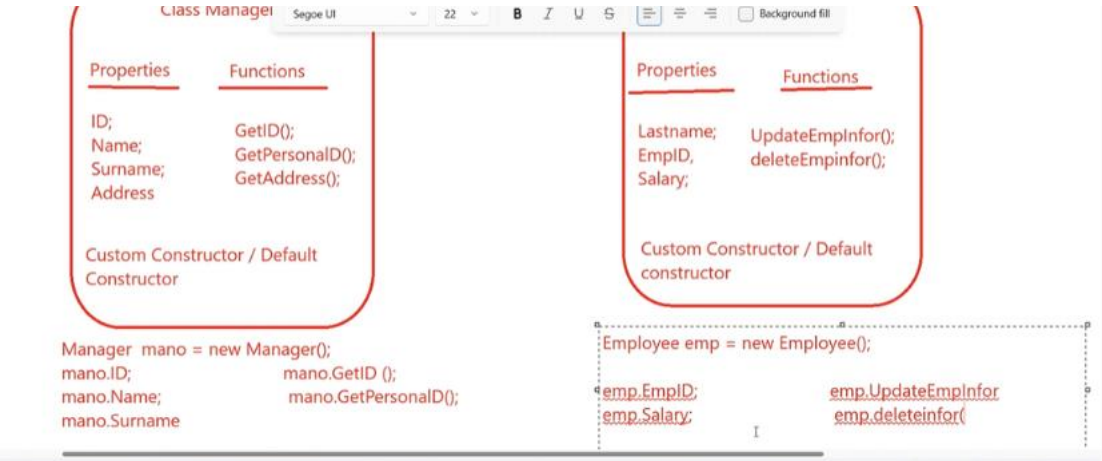
Example

Get yo

Create a constructor:

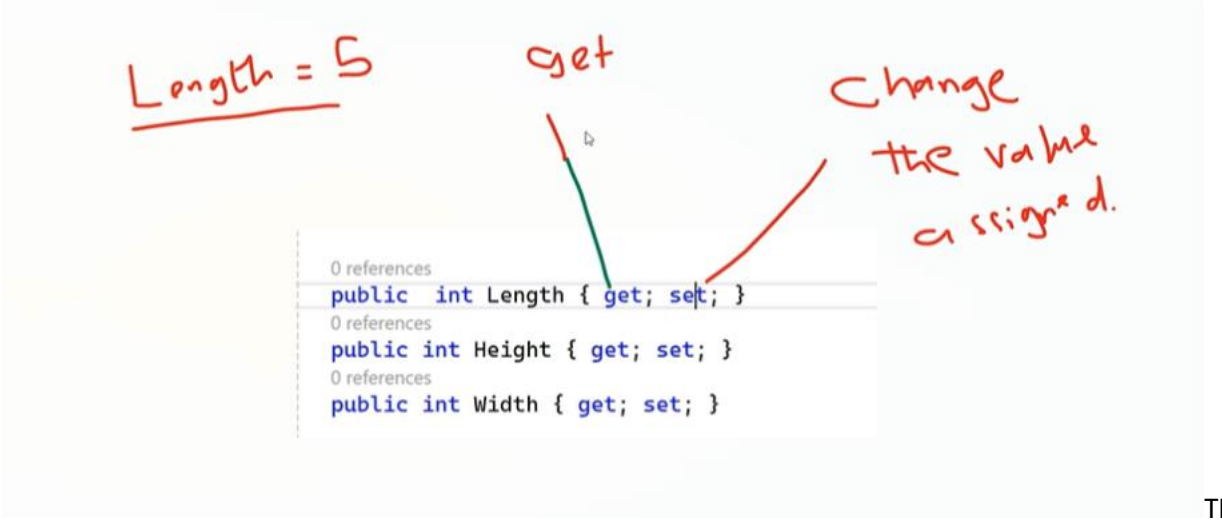
```
// Create a Car class
class Car
{
  public string model;  // Create a field

  // Create a class constructor for the Car class
  public Car()
  {
    model = "Mustang"; // Set the initial value for model
  }

  static void Main(string[] args)
  {
    Car Ford = new Car();  // Create an object of the Car Class (this will call the constructor)
    Console.WriteLine(Ford.model);  // Print the value of model
  }
}

// Outputs "Mustang"
```

WEDNESDAY (25/FEB/2026) - **[WEEK4]**



- Day 3 Taks

The get-set, gets the value then sets the value



The Square class is a parent class, and rectangle is a child class, this means it is **inheriting** the properties of the Square:

```
Rectangle.cs*   Team Explorer - Home   Git Changes - demo2   Square.cs*   Program.cs*
ConsoleApp5                            ConsoleApp5.Rectangle
1   using System;
2   using System.Collections.Generic;
3   using System.Linq;
4   using System.Text;
5   using System.Threading.Tasks;
6
7   namespace ConsoleApp5
8   {
        0 references
9       public class Rectangle : Square
10      {
11      }
12  }
13
```

```
using ConsoleApp5;

Rectangle rec = new Rectangle();

rec.CalculateVolume(5,10,10);

rec.Width = 10;
rec.Height = 10;
```

The **Access modifiers** are a principle of **Encapsulation** (To hide things)

-Public

-Private

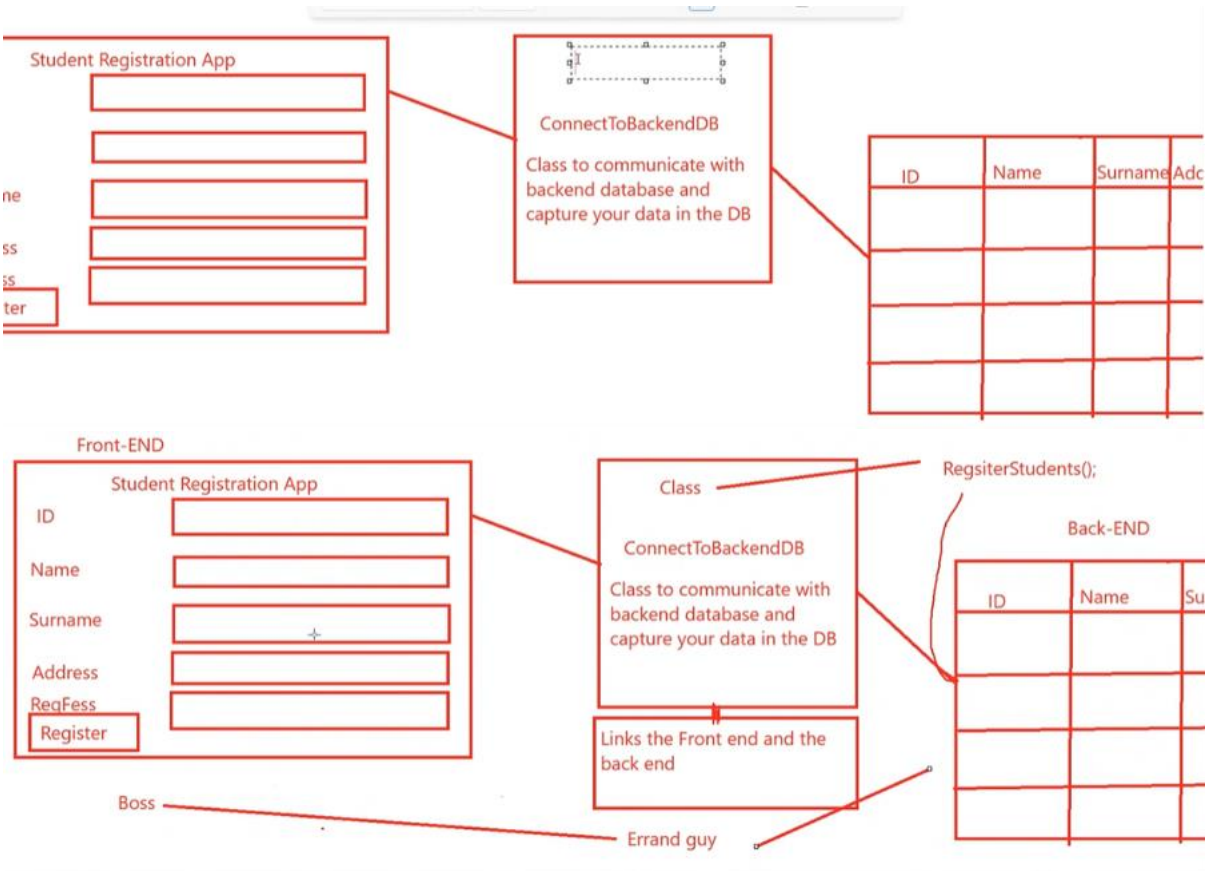
-Protected

*Sealed is a special Keyword that prevents access to the class, the class in not supposed to be inherited.

*Abstract, shows the Parent class is not complete, allows for changes to be made even after the inheritance of another subclass (child class).

***Polymorphism**, Uniqueness for a subclass (child class) has. Means the parent class does not have these properties, they're unique to the child class.

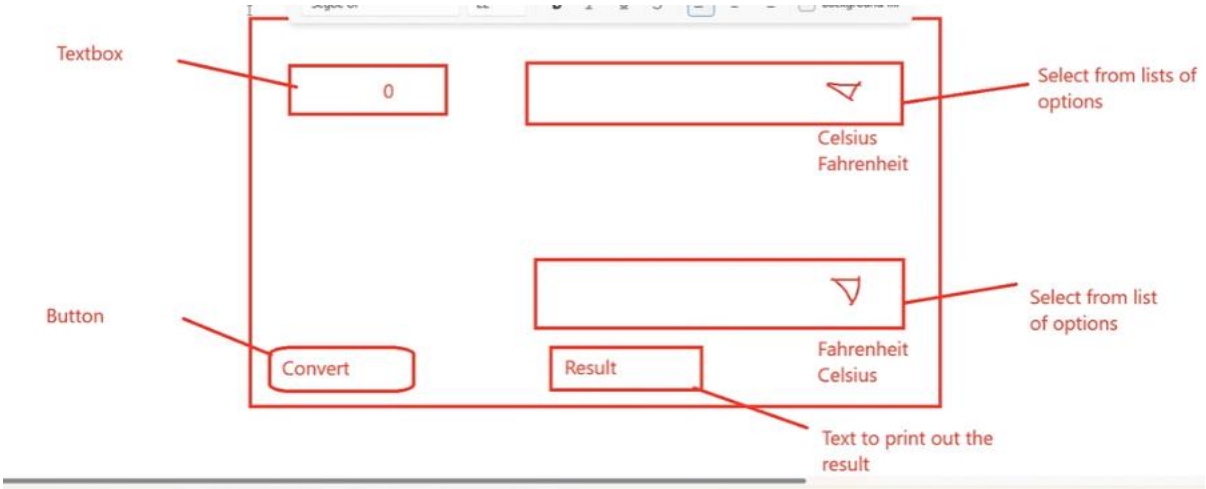
*We're making student registration application



*Another Class Activity

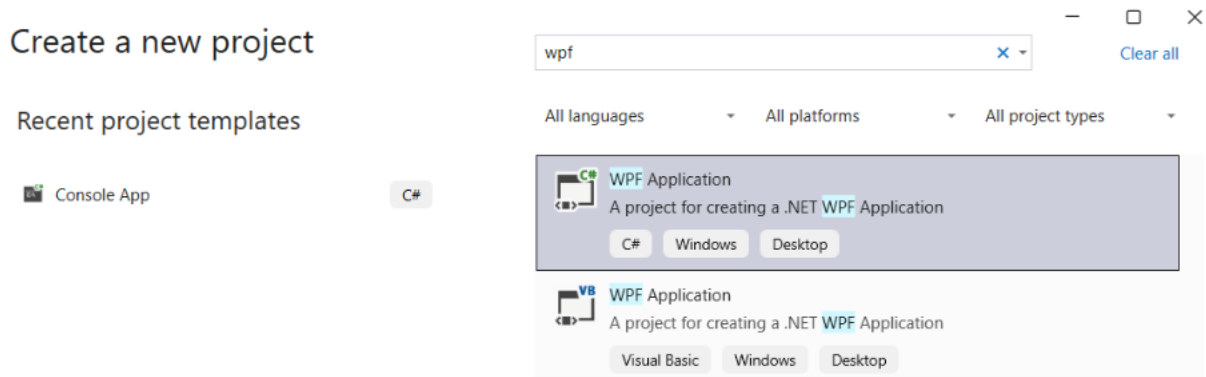
THURSDAY (26/FEB/2026) - **[WEEK4]**

We're building a temperature conversion program:



-Combo box

-Windows Presentation Foundation (WPF)



HERE's The class project (Temperature converter App):
* We customized all the elements or object in our WPF App from text to background colour

```
0 references
public MainWindow()
{
    InitializeComponent();


    string[] a = { "Celsius", "Fahrenheit" };
    string[] b = { "Fahrenheit", "Celsius" };

    combobox1.ItemsSource = a;
    combobox2.ItemsSource = b;
}
```

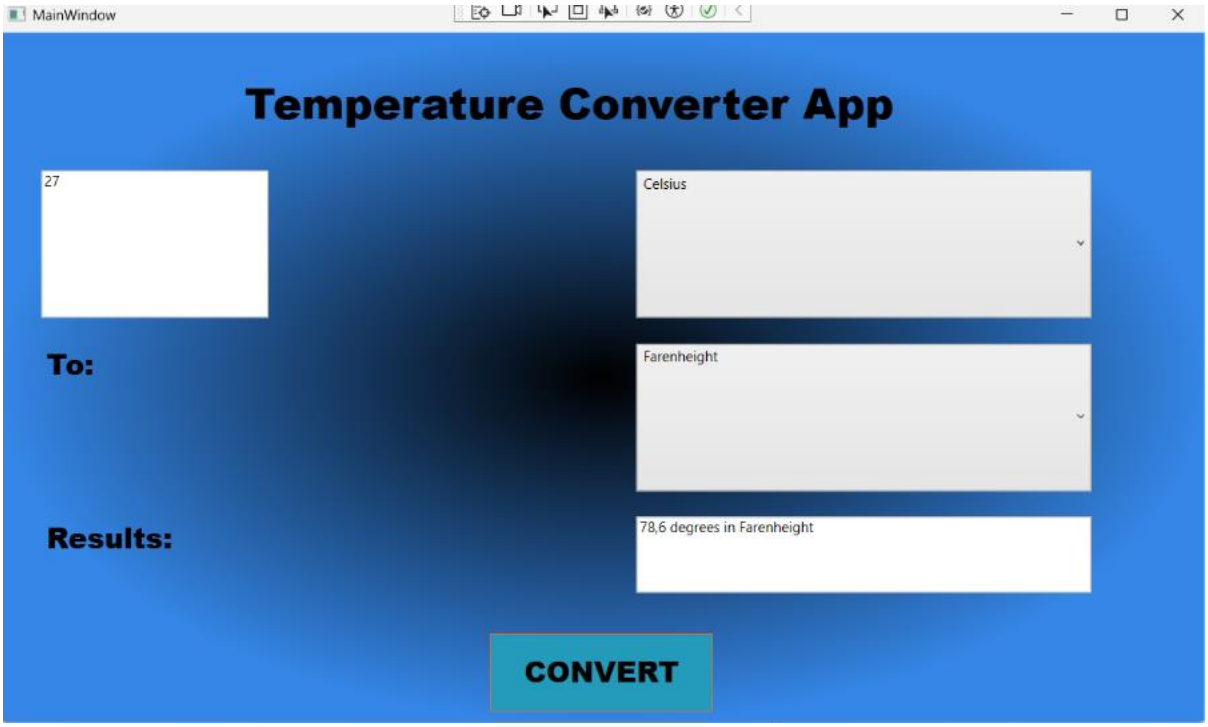
```
double startingTemp;
double resultTemp;

startingTemp = double.Parse(txtinput.Text);

if(combobox1.Text == "Celsius" && combobox2.Text == "Fahrenheit")
{
    resultTemp = (startingTemp * (1.8 + 32));
    txtresults.Text = resultTemp.ToString() + "degrees in Celsius";
}
if(combobox1.Text == "Fahrenheit" && combobox2.Text == "Celsius")
{
    resultTemp = ((startingTemp - 32) / 1.8);
    txtresults.Text = resultTemp.ToString() + "degrees in Fahrenheit";
}
```

```
using System.Text;

using System.Windows;

using System.Windows.Controls;

using System.Windows.Data;

using System.Windows.Documents;

using System.Windows.Input;

using System.Windows.Media;

using System.Windows.Media.Imaging;

using System.Windows.Navigation;

using System.Windows.Shapes;


namespace Prj_TemperatureConverters

{

   /// <summary>

   /// Interaction logic for MainWindow.xaml

   /// </summary>

   public partial class MainWindow : Window
```

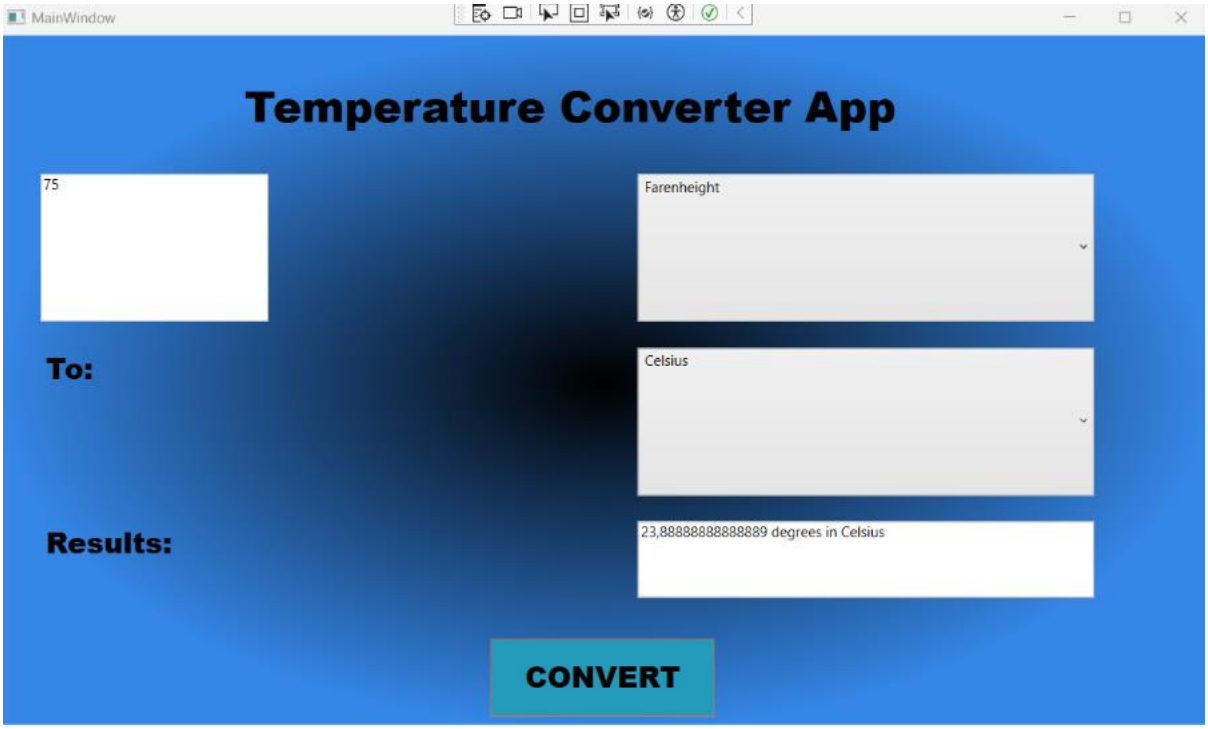
```
{
    public MainWindow()
    {
        InitializeComponent();
        string [] a = {"Celsius", "Fahrenheit" }; //List of options for the combo boxes
        string[] b = { "Fahrenheit", "Celsius" };

        cmbb1.ItemsSource = a; //assignment of options into the specific combo box
        cmbb2.ItemsSource = b;
    }

    //The main button that controls the program
    private void Button_Click(object sender, RoutedEventArgs e)
    {



        double startTemp = double.Parse(txbinput.Text);// variable startTemp will know to accept a
double value from user
        double resultTemp;

        if (cmbb1.Text == "Celsius" && cmbb2.Text == "Fahrenheit") {
            resultTemp = ((startTemp * 1.8) + 30); //Mathematical Formula to convert Celsius into
Farenheight
            txbresult.Text = resultTemp.ToString() + " degrees in Fahrenheit ";

        }
        if (cmbb1.Text == "Fahrenheit" && cmbb2.Text == "Celsius")
        {
```

```
            resultTemp = ((startTemp - 32)/ 1.8);  //Mathematical Formula to convert Farenheight into
Celsius

            txbresult.Text = resultTemp.ToString() + " degrees in Celsius";

        }

        if (cmbb1.Text == "Celsius" && cmbb2.Text == "Celsius")

        {

           //Mathematical Formula to convert Farenheight into Celsius

            txbresult.Text = startTemp.ToString() + " degrees in Celsius";

        }

        if (cmbb1.Text == "Fahrenheit" && cmbb2.Text == "Fahrenheit")

        {

            //Mathematical Formula to convert Farenheight into Celsius

            txbresult.Text = startTemp.ToString() + " degrees in Fahrenheit ";

        }




      }

   }

}
```

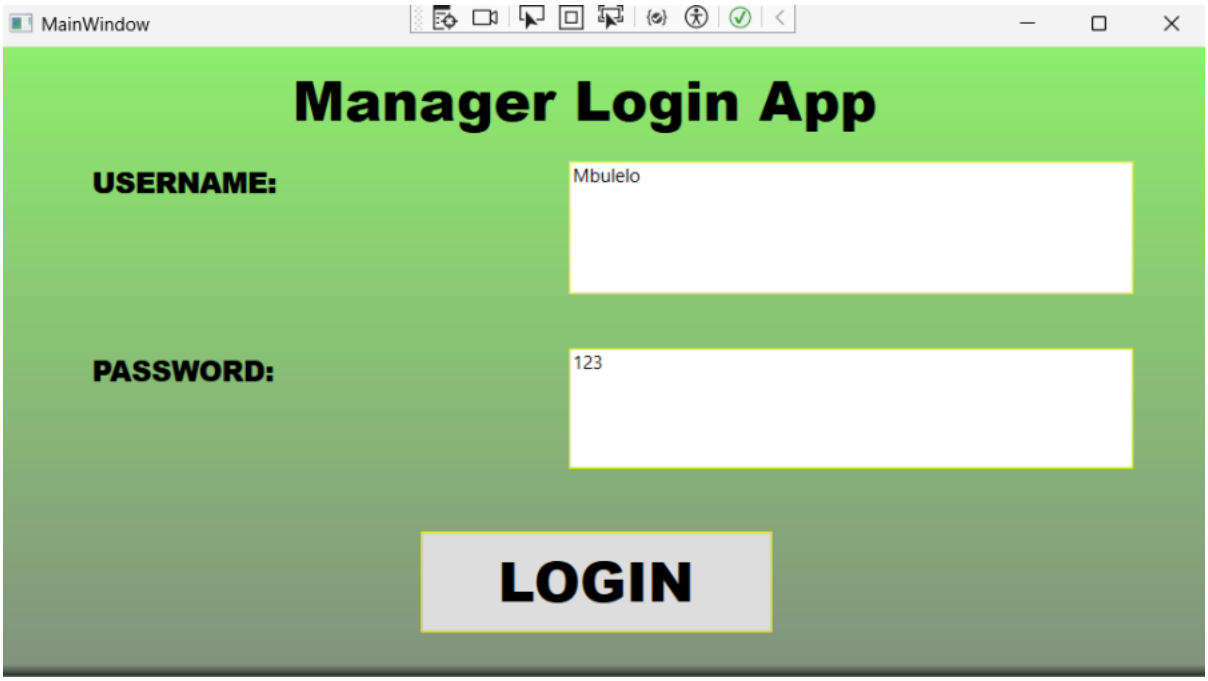
Result:
(Celsius to Fahrenheit)

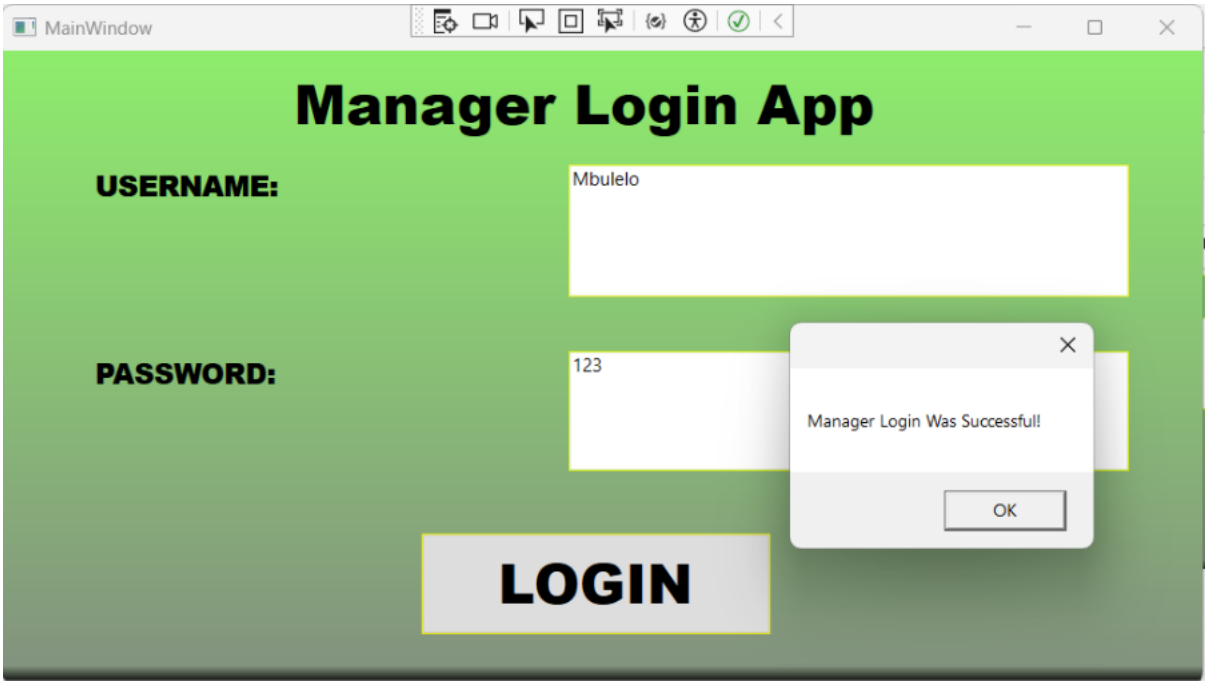


(Fahrenheit to Celsius):

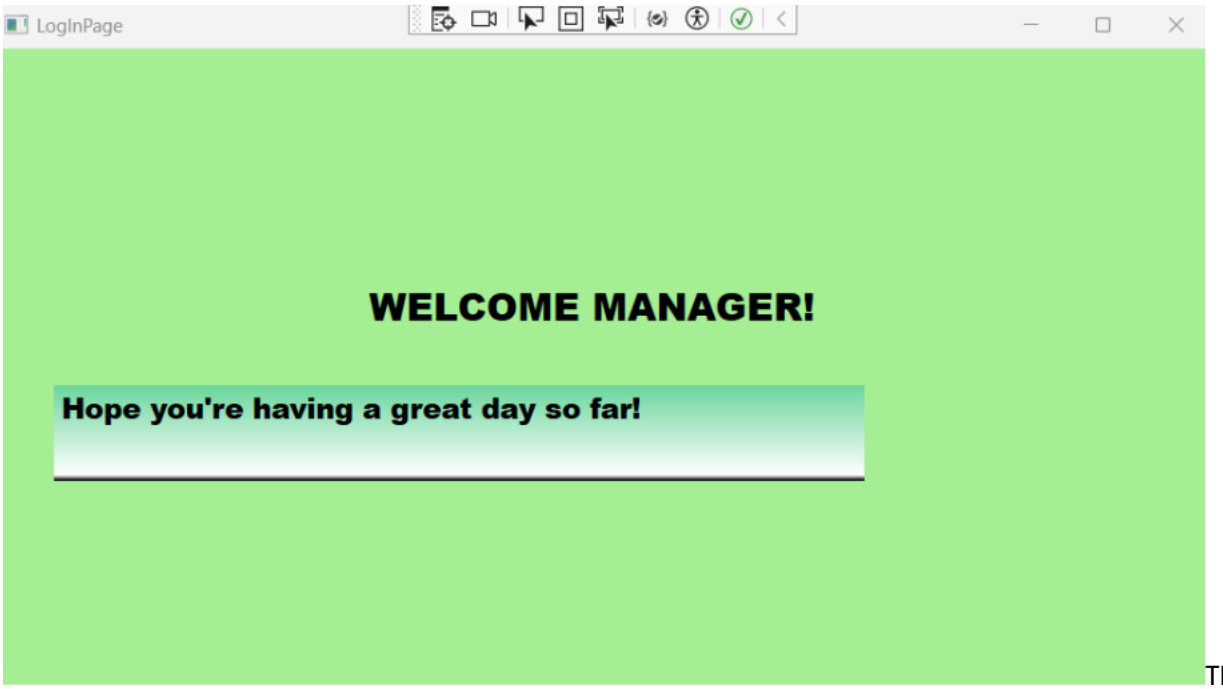


Second App (Managers):





Then leads to:



The code:

```
using System.Text;

using System.Windows;

using System.Windows.Controls;

using System.Windows.Data;

using System.Windows.Documents;

using System.Windows.Input;

using System.Windows.Media;

using System.Windows.Media.Imaging;

using System.Windows.Navigation;

using System.Windows.Shapes;


namespace ManagersLogin

{

   /// <summary>

   /// Interaction logic for MainWindow.xaml
```

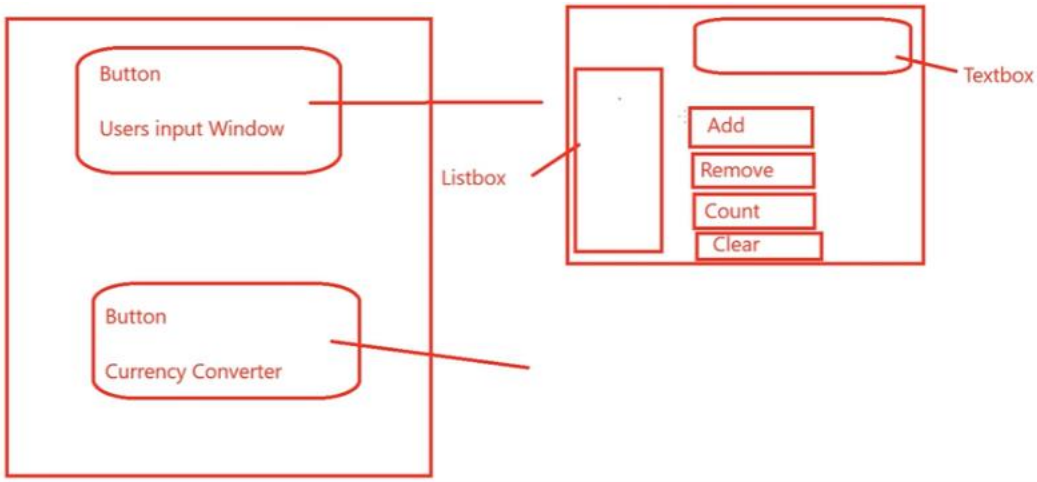
```
    /// </summary>
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void Button_Click(object sender, RoutedEventArgs e)
        {
            string username1 = usrname1.Text;
            string password1 = pssword.Text;

            if (username1 ==  "Mbulelo" && password1 == "123") {
                MessageBox.Show("Manager Login Was Successful!");

                LogInPage nav = new LogInPage();
                nav.Show();
               // LogInPage Displays
            }
            else {
                MessageBox.Show("Incorrect Username or Password");
            }
        }
    }
}
```

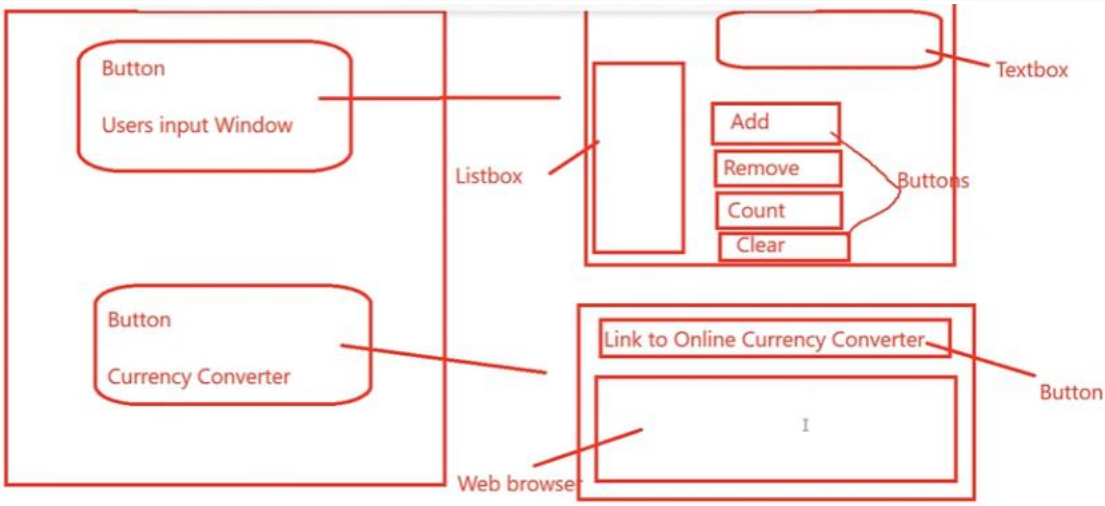
FRIDAY (27/FEB/2026) - **[WEEK4]**

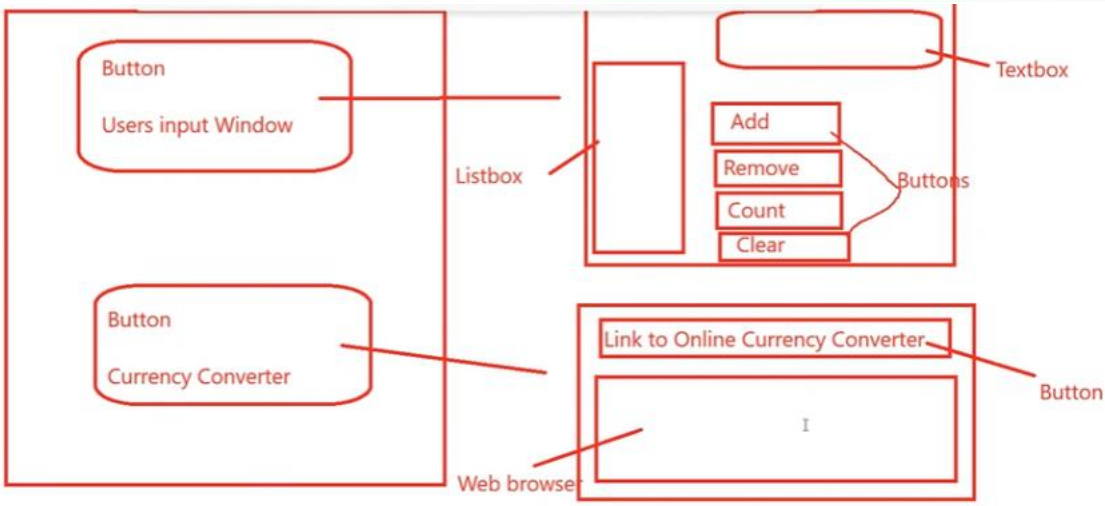
*Github

*Navigations App

*App To Make (Exchange rates & Currency converters)



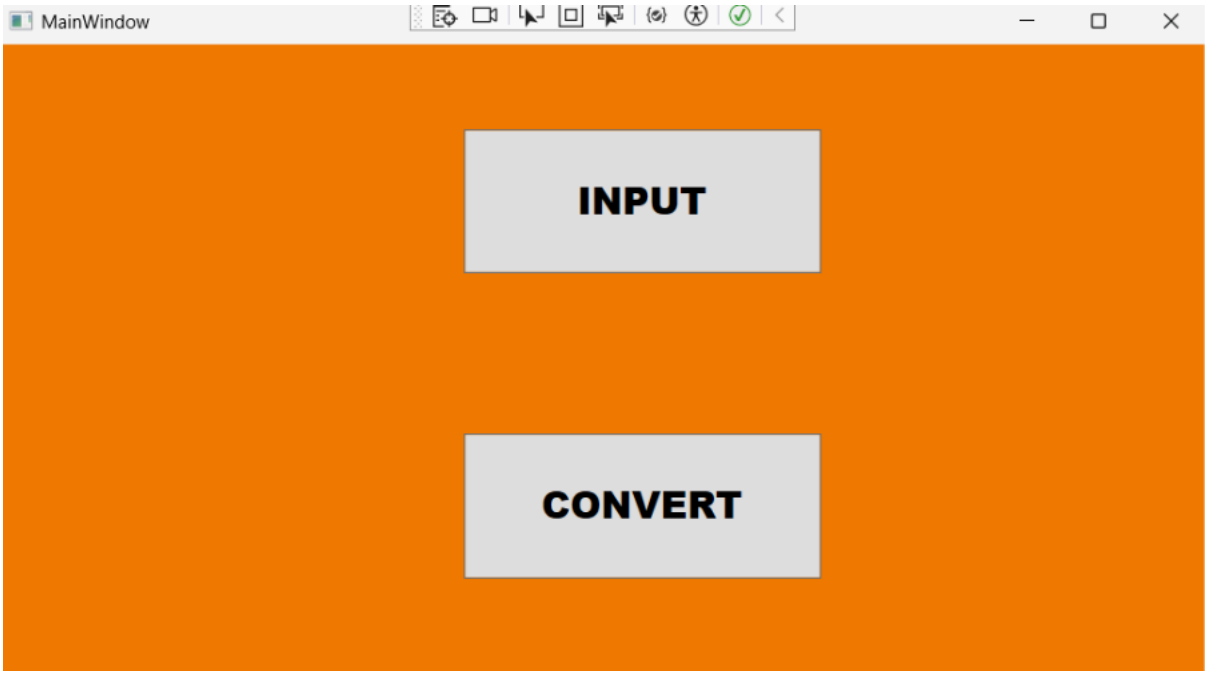


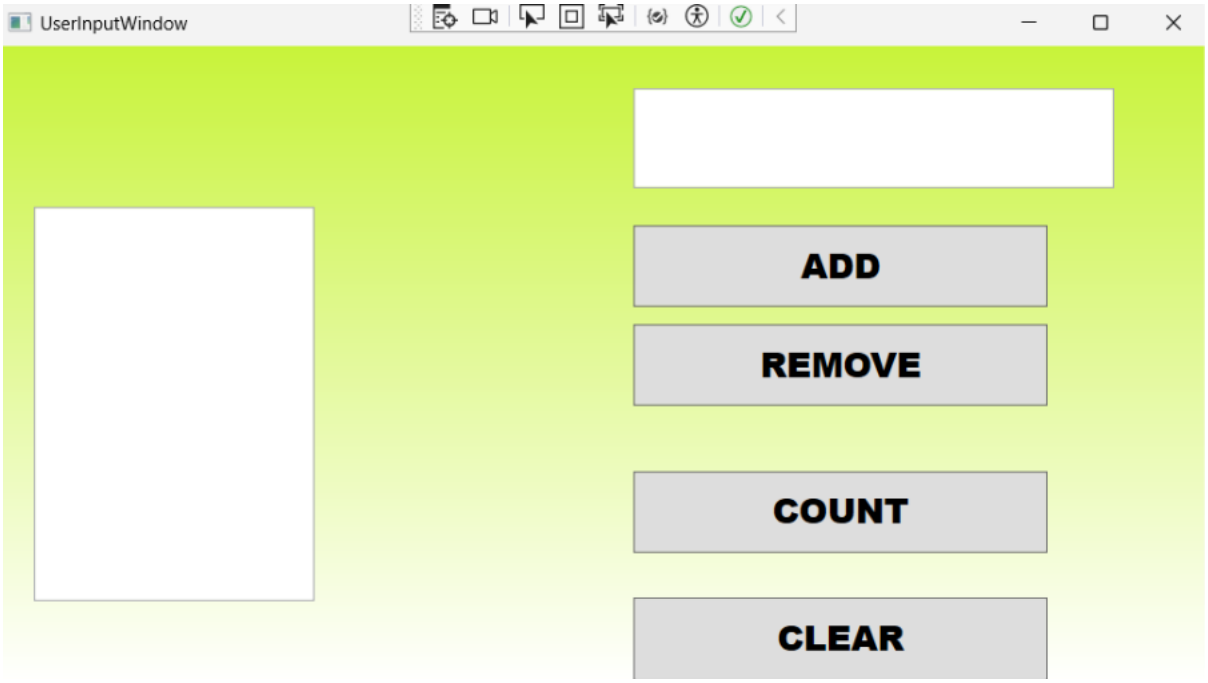


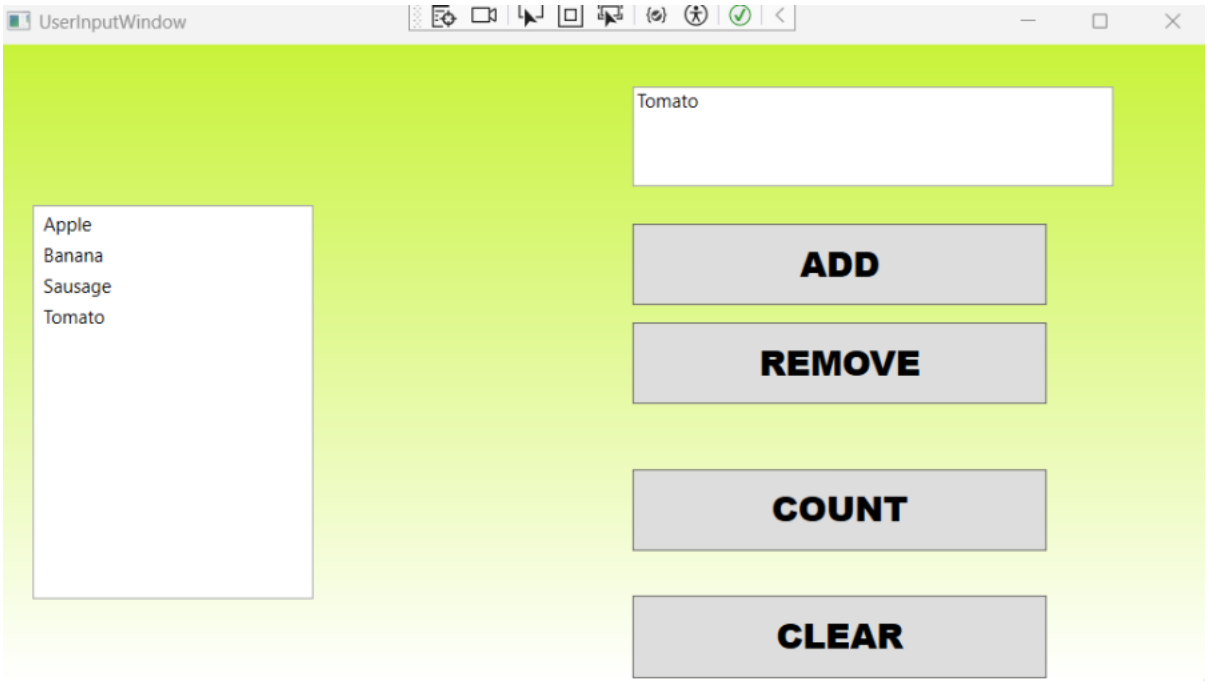

```
using System;

using System.Collections.Generic;

using System.Text;
```

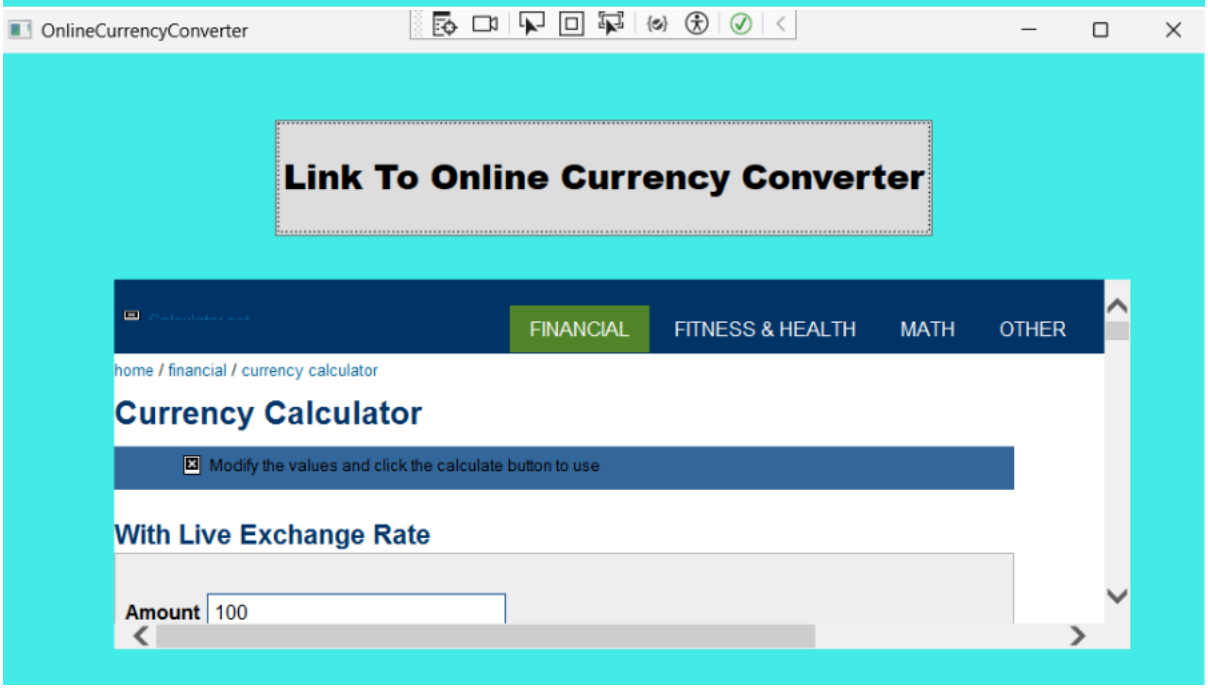
```
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Shapes;

namespace currencyConverter
{
    /// <summary>
    /// Interaction logic for UserInputWindow.xaml
    /// </summary>
    public partial class UserInputWindow : Window
    {
        public UserInputWindow()
        {
            InitializeComponent();
        }

        //Clear
        private void Button_Click(object sender, RoutedEventArgs e)
        {
            userlist.Items.Clear();
        }
```

```
        //Add Items
        private void useradd_Click(object sender, RoutedEventArgs e)
        {
            userlist.Items.Add(usertxtbox.Text);


        }


        //Removes
        private void userremove_Click(object sender, RoutedEventArgs e)
        {
            this.userlist.Items.RemoveAt(userlist.SelectedIndex);
        }


        //Count Function
        private void usercount_Click(object sender, RoutedEventArgs e)
        {
            MessageBox.Show(this.userlist.Items.Count.ToString(), " Current Value");
        }
    }
}
```


MainWindow
INPUT
CONVERT


UserInputWindow
ADD
REMOVE
COUNT
CLEAR


UserInputWindow
Tomato
Apple
Banana
Sausage
Tomato
ADD
REMOVE
COUNT
CLEAR




```
using System;

using System.Collections.Generic;

using System.Text;

using System.Windows;

using System.Windows.Controls;

using System.Windows.Data;
```

```
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Shapes;

namespace currencyConverter
{
    /// <summary>
    /// Interaction logic for OnlineCurrencyConverter.xaml
    /// </summary>
    public partial class OnlineCurrencyConverter : Window
    {
        public OnlineCurrencyConverter()
        {
            InitializeComponent();
        }

        private void occBtn_Click(object sender, RoutedEventArgs e)
        {
            webb.Navigate("https://www.calculator.net/currency-calculator.html");

        }
    }
}
```